

UNIT I

Operating System Overview: Introduction, Operating System functions, Operating System operations, Computing environments, Free and Open-Source Operating Systems
System Structures: Operating System Services, User and Operating-System Interface, System Calls, Types of System Calls, System programs, Operating System Design and Implementation, Operating System structure, Building and Booting an Operating System, Operating System debugging.

Part-1

Operating System Overview:

1.Introduction:

- A program that acts as an intermediary between a user of a computer and the computer hardware
- It manages the computer hardware.

Operating system goals:

- Execute user programs and make solving user problems easier
- Make the computer system convenient to use
- Use the computer hardware in an efficient manner

2.Operating System functions:

Operating systems perform a wide range of functions to manage computer resources efficiently.

Process management:

- Manages the CPU by scheduling which processes get to run on the processor and when. This includes managing process creation, termination, and synchronization.
- A program is passive (stored on disk); a process is active (program in execution).
- A process needs resources: CPU time, memory, files, and I/O devices.
- Processes may receive input data (e.g., a browser gets a URL) When a process ends, the OS reclaims resources.
- A single-threaded process has one program counter; execution is sequential.
- Two processes can run from the same program, but each is a separate execution.
- A multithreaded process has multiple program counters (one per thread).
- Processes are the unit of work in a system.

Process Management Activities

The operating system is responsible for the following activities in connection with process management:

- Creating and deleting both user and system processes
- Suspending and resuming processes
- Providing mechanisms for process synchronization
- Providing mechanisms for process communication
- Providing mechanisms for deadlock handling

Memory management:

- Manages the computer's main memory (RAM) by allocating and deallocating memory to processes to ensure efficient use.
- Main memory is central to computer operation; it is a large array of bytes, each with a unique address.
- Memory is shared by the CPU and I/O devices.

Memory management activities

- Keeping track of which parts of memory are currently being used and by whom
- Deciding which processes (or parts thereof) and data to move into and out of memory
- Allocating and deallocating memory space as needed

File management:

- Manages the storage and organization of files and directories on secondary storage devices like hard disks. This includes operations like creating, deleting, and modifying files.
 - Files usually organized into directories
 - Access control on most systems to determine who can access what
- The OS gives users a **logical, uniform view** of storage by abstracting physical devices into **files**.

File-System management activities include

- Creating and deleting files and directories
- Primitives to manipulate files and directories
- Mapping files onto secondary storage
- Backup files onto stable (non-volatile) storage media

Mass Storage Management:

- Systems require **secondary storage** to back up main memory.
- Modern computers mainly use **HDDs** and **NVM devices** (like SSDs) for program and data storage.
- Programs (compilers, browsers, games, etc.) reside on these devices until loaded into memory.
- During execution, programs read and write data to these storage devices.
- Therefore, **managing secondary storage efficiently is essential** for system performance and reliability.
- The operating system is responsible for the following activities in connection with secondary storage management:
 - Mounting and un-mounting
 - Free-space management
 - Storage allocation
 - Disk scheduling
 - Partitioning
 - Protection

Cache Management:

- **Caching** stores frequently used data in a **faster storage** (cache) for quick access.
- When data is needed:
 - Check cache first.
 - If present → use it.
 - If absent → fetch from main memory/disk and place into cache.
- **Registers** act as the fastest cache; compilers decide what stays in registers.
- Hardware commonly provides:
 - **Instruction caches**
 - **Data caches**
(OS does not manage these.)
- Because cache size is limited, **replacement policies** are needed to decide which data to evict.

- Data may exist at **multiple levels**: disk → memory → cache → registers.
 - Values differ temporarily until updates are written back.

I/O System Management:

- Manages all the input and output operations of the computer, ensuring data is transferred correctly between the CPU and peripherals.
- OS hides hardware device details from users.
- In UNIX and similar systems, this is done through the **I/O subsystem**.
- The I/O subsystem consists of several components:
 - A memory-management component that includes buffering, caching, and spooling
 - A general device-driver interface
 - Drivers for specific hardware devices
 - Only the **device driver** knows the exact details of its hardware device.

Security & Protection Management:

- Protects the system and its data by controlling access through features like user accounts and passwords, and by preventing unauthorized access to system resources.
 - to protect system resources, data, and users from unauthorized access, breaches, and other threats.
 - User authentication: Login screens requiring usernames and passwords.
 - File permissions: Allowing read, write, or execute rights only to authorized users.
 - Firewalls and Network Security
 - Malware and Threat Protection
 - Security Updates and Patching

Error detection and handling:

- Detects and handles errors, logs them, and provides information to the user so that the system can continue to operate properly.
 - If an application crashes, the operating system detects this error, terminates the problematic process, and often provides an error message to the user.
 - In more serious cases, like a system-wide crash, the OS might initiate a controlled shutdown and restart to prevent data loss.

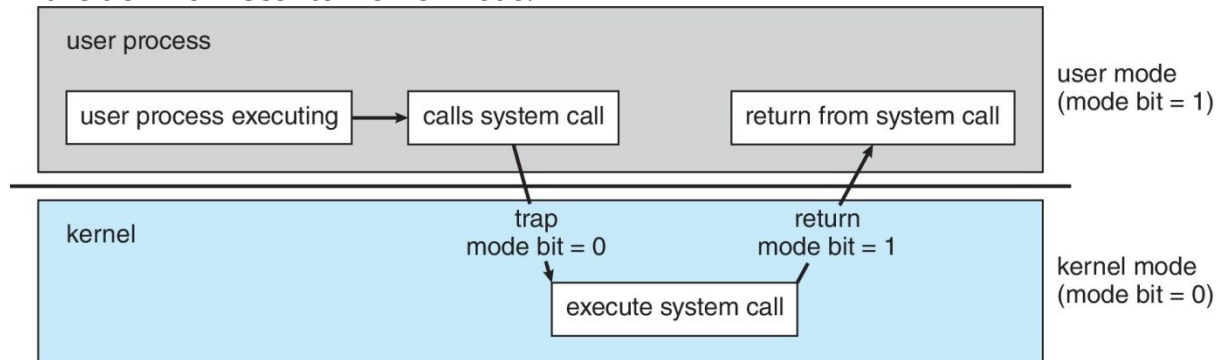
3. Operating System operations:

Dual-Mode Operation:

- **Dual-mode** operation allows OS to protect itself and other system components
 - **User mode** and **kernel mode**
- **Mode bit** provided by hardware
 - Provides ability to distinguish when system is running user code or kernel code.
 - When a user is running ⇒ mode bit is “user”
 - When kernel code is executing ⇒ mode bit is “kernel”
- How do we guarantee that user does not explicitly set the mode bit to “kernel”?
 - System call changes mode to kernel, return from call resets it to user
- Some instructions designated as **privileged**, only executable in kernel mode
- **user mode**: Applications run with limited access to system resources, ensuring stability and security. Also called **restricted mode**.

- **kernel mode:** the OS gains unrestricted access to hardware resources for managing processes and memory. Also called **System mode**, **Supervisor Mode**, or **Privileged Mode**.
- **Dual mode operation** is a critical concept that ensures the system's **stability, security, and efficient resource management**.

Transition from User to Kernel Mode:



Multimode Operation:

Some operating systems and processor architectures support multiple modes of operation. (ARM TrustZone, Intel SGX)

Modern operating systems typically support at least three modes of operation:

- **User Mode:** Least privileged, where most applications run.
- **Kernel Mode:** Highest privilege, where the core OS functions operate.
- **Supervisor Mode:** An intermediate mode between user and kernel modes which is used to manage virtual machines.

These CPU's have a separate mode that specifies when the virtual machine manager (VMM) and the virtualisation management software is in control of the system.

For these systems, the virtual mode has more privileges than user mode but less than kernel mode.

4.Computing environments:

Traditional:

- Stand-alone general-purpose machines
- most systems interconnect with others (i.e., the Internet)
- **Portals** provide web access to internal systems
- **Network computers (thin clients)** are like Web terminals
- Mobile computers interconnect via **wireless networks**
- Networking becoming ubiquitous – even home systems use **firewalls** to protect home computers from Internet attacks



Mobile:

- Handheld smartphones, tablets, etc.
- the functional difference between them and a “traditional” laptop? Extra feature – more OS features (GPS, gyroscope)
- Allows new types of apps like *augmented reality*
- Use IEEE 802.11 wireless, or cellular data networks for connectivity
- Leaders are **Apple iOS** and **Google Android**



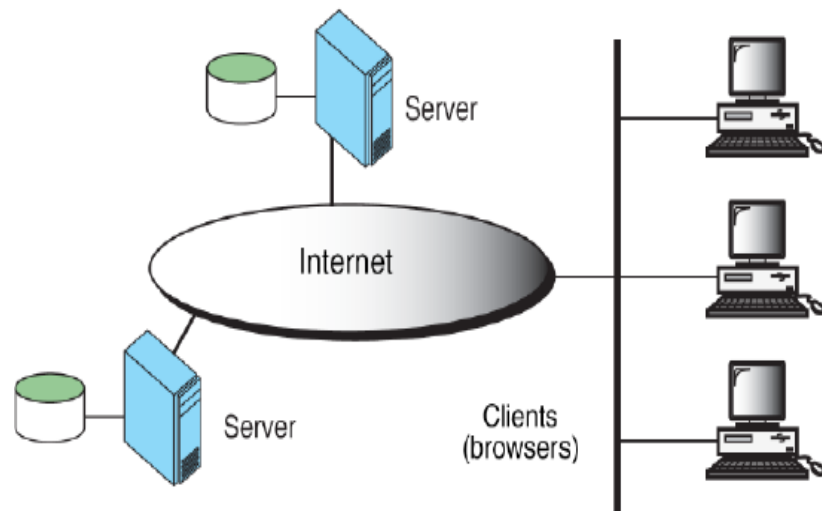
Distributed:

Collection of separate, possibly heterogeneous, systems networked together

- **Network** is a communications path, **TCP/IP** most common
- **Local Area Network (LAN)** – Office LAN
- **Wide Area Network (WAN)** – Corporate WAN (different offices)
- **Metropolitan Area Network (MAN)** – free Wi-Fi across city
- **Personal Area Network (PAN)** – Smartphone to Smartwatch

Network Operating System provides features between systems across network

- Communication scheme allows systems to exchange messages
- Illusion of a single system



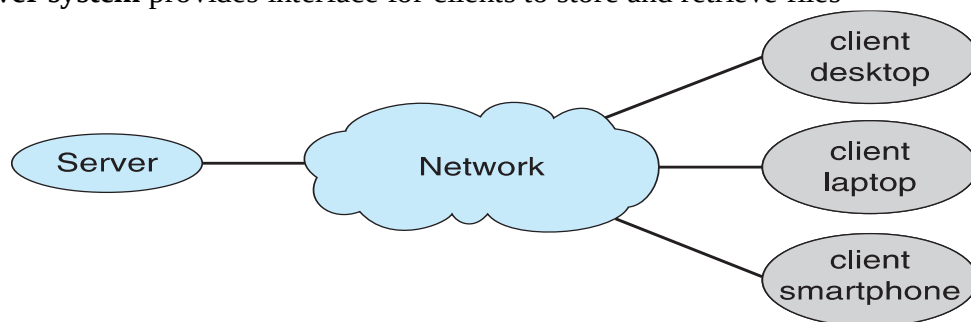
Client Server:

Client-Server Computing Dumb terminals supplanted by smart PCs

Many systems now **servers**, responding to requests generated by **clients**

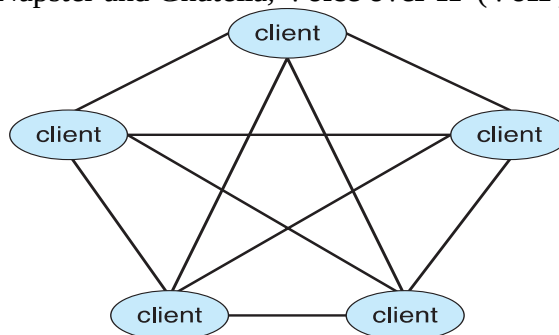
Compute-server system provides an interface to client to request services (i.e., database)

File-server system provides interface for clients to store and retrieve files



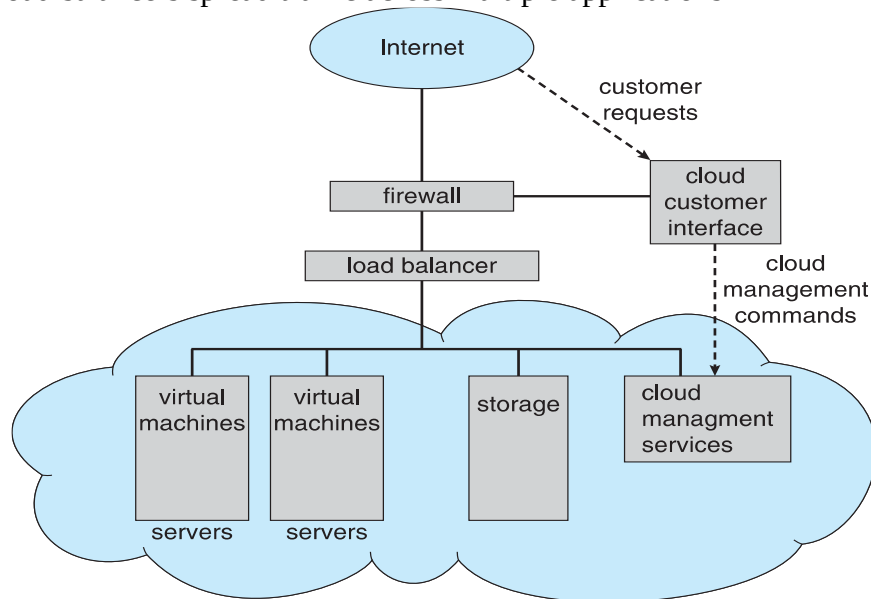
Peer-to-Peer:

- Another model of distributed system
- P2P does not distinguish clients and servers
 - Instead all nodes are considered peers
 - May each act as client, server or both
 - Node must join P2P network
 - ▶ Registers its service with central lookup service on network, or
 - ▶ Broadcast request for service and respond to requests for service via **discovery protocol**
- Examples include Napster and Gnutella, **Voice over IP (VoIP)** such as Skype



Cloud computing:

- Cloud computing environments composed of traditional OSes, plus VMMs, plus cloud management tools
 - Internet connectivity requires security like firewalls
 - Load balancers spread traffic across multiple applications



Real-time Embedded:

- Real-time embedded systems most prevalent form of computers
 - Vary considerable, special purpose, limited purpose OS, **real-time OS**
 - Use expanding
- Many other special computing environments as well
 - Some have OSes, some perform tasks without an OS
- Real-time OS has well-defined fixed time constraints
 - Processing **must** be done within constraint
 - Correct operation only if constraints met

5. Free and Open-Source Operating Systems:

Free Software vs. Open-Source Software has Same goals

Free software focuses on user freedom and ethical principles.

Open-source software emphasizes practicality, collaborative development, and better code quality.

Examples:

GNU/Linux: A family of operating systems using the Linux kernel and GNU tools.

- Examples: Ubuntu, Fedora, Debian

BSD UNIX: Berkeley Software Distribution, a UNIX derivative.

- Examples: FreeBSD, OpenBSD, NetBSD

Android: Google's mobile operating system (based on a modified Linux kernel)

Virtual Machine Monitors (VMM) or virtualization software allows users to run multiple operating systems on one physical machine. Examples include:

VMware Player: A free virtualization tool for Windows.

VirtualBox: Open-source and free, available for multiple platforms like Windows, macOS, and Linux.

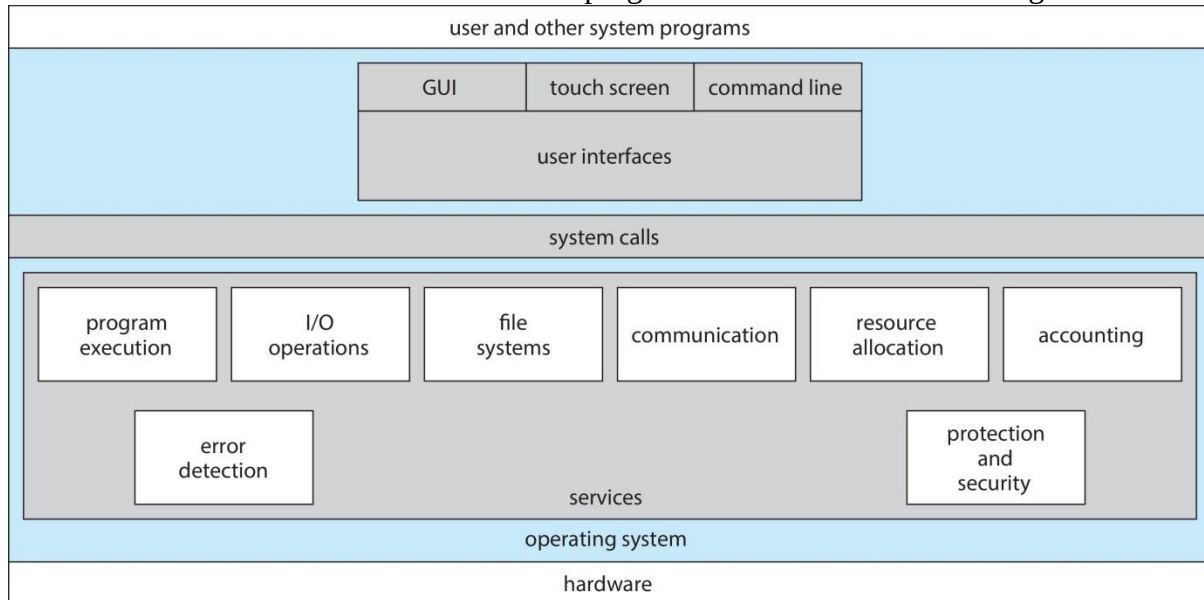
Useful for learning, testing, and experimenting with different operating systems without the need for dedicated hardware.

Part-2

System Structures:

1. Operating System Services:

- Operating systems provides an environment for the execution of programs.
- It makes certain services available to programs and to users of those Programs.



A View of Operating System Services

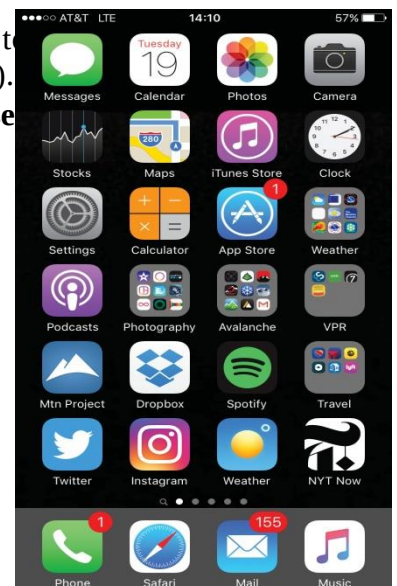
```

rankunar@trusty:~$ pwd
/home/rankunar
rankunar@trusty:~$ pushd .
~ ~
rankunar@trusty:~$ pushd /opt/ros/
/opt/ros ~ ~
rankunar@trusty:/opt/ros$ pushd /opt/chefdk/
/opt/chefdk /opt/ros ~ ~
rankunar@trusty:/opt/chefdk$ pwd
/opt/ros ~ ~
rankunar@trusty:/opt/ros$ pwd
/opt/ros
rankunar@trusty:/opt/ros$ pwd
/opt/ros$pwd
~ ~
rankunar@trusty:~$ pwd
/home/rankunar
rankunar@trusty:~$ cd /
rankunar@trusty:~$ pwd
rankunar@trusty:~$
rankunar@trusty:~$ pwd
/home/rankunar
rankunar@trusty:~$
  
```

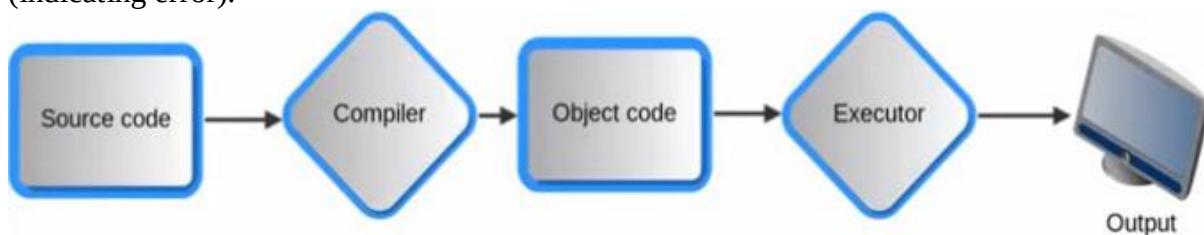
in services
operating
command-L



ful t
UI).
Use

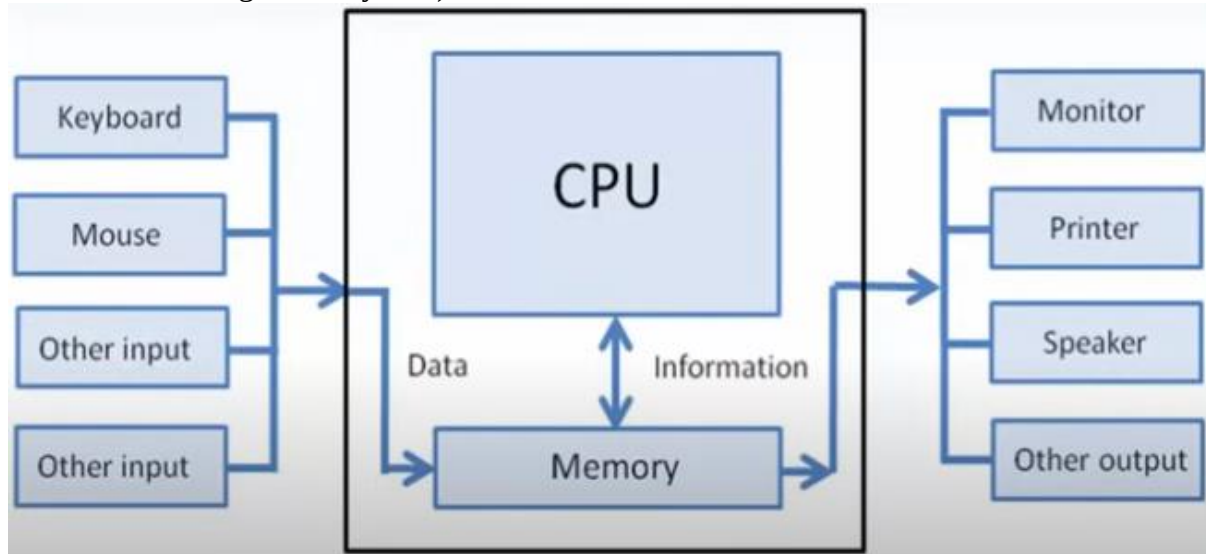


Program execution - The system must be able to load a program into memory and to run that program. The program must be able to end its execution, either normally or abnormally (indicating error).



I/O operations - A running program may require I/O, which may involve a file or an I/O device.

For specific devices, special functions may be desired (such as reading from a network interface or writing to a file system).



File-system manipulation - The file system is of particular interest. Obviously, programs need to read and write files and directories.

They also need to create and delete them by name, search for a given file, and list file information.

Finally, some operating systems include permissions management to allow or deny access to files or directories based on file ownership.

Communications - Processes may exchange information, on the same computer or between computers over a network

Communications may be via shared memory or through message passing (packets moved by the OS)

Error detection - OS needs to be constantly aware of possible errors

- May occur in the CPU and memory hardware, in I/O devices, in user program
- For each type of error, OS should take the appropriate action to ensure correct and consistent computing
- Debugging facilities can greatly enhance the user's and programmer's abilities to efficiently use the system

Another set of OS services exists for ensuring the efficient operation of the system itself via resource sharing

Resource allocation - When multiple users or multiple jobs running concurrently, resources must be allocated to each of them

- Many types of resources - CPU cycles, main memory, file storage, I/O devices.

Accounting - To keep track of which users use how much and what kinds of computer resources.

- This record keeping may be used for accounting (so that users can be billed) or simply for accumulating usage statistics.

Protection and security - The owners of information stored in a multiuser or networked computer system may want to control use of that information, concurrent processes should not interfere with each other

- **Protection** involves ensuring that all access to system resources is controlled

- **Security** of the system from outsiders requires user authentication, extends to defending external I/O devices from invalid access attempts

2. User and Operating-System Interface:

Command Line interpreter:

- CLI allows direct command entry
- Sometimes implemented in kernel, sometimes by systems program
- Sometimes multiple flavors implemented – the interpreters are known as **shells**(*C shell, Bourne-Again shell, Korn shell, Bourne shell*)
- Primarily fetches a command from user and executes it.
- In one approach, the command interpreter itself contains the code to execute the command.
- An alternative approach, the command interpreter does not understand the command in any way; it merely uses the command to identify a file to be loaded into memory and executed.
- If the latter, adding new features doesn't require shell modification

```

1. root@r6181-d5-us01:~ (ssh)
Last login: Thu Jul 14 08:47:01 on ttys002
iMacPro:~ pbg$ ssh root@r6181-d5-us01
root@r6181-d5-us01's password:
Last login: Thu Jul 14 06:01:11 2016 from 172.16.16.162
[root@r6181-d5-us01 ~]# uptime
 06:57:48 up 16 days, 10:52,  3 users,  load average: 129.52, 80.33, 56.55
[root@r6181-d5-us01 ~]# df -kh
Filesystem                Size      Used Avail Use% Mounted on
/dev/mapper/vg_ks-lv_root    50G       19G   28G   41% /
tmpfs                      127G      520K   127G    1% /dev/shm
/dev/sda1                   477M       71M   381M   16% /boot
/dev/dssd0000               1.0T     480G   545G   47% /dssd_xfs
tcp://192.168.150.1:3334/orangefs 12T     5.7T   6.4T   47% /mnt/orangefs
/dev/gpfs-test              23T     1.1T   22T    5% /mnt/gpfs
[root@r6181-d5-us01 ~]#
[root@r6181-d5-us01 ~]# ps aux | sort -nrk 3,3 | head -n 5
root    97653 11.2  6.6 42665344 17520636 ?    S<Ll  Jul13 166:23 /usr/lpp/mmfs/bin/mmfdsd
root    69849  6.6  0.0      0      0 ?        S    Jul12 181:54 [vpthread-1-1]
root    69850  6.4  0.0      0      0 ?        S    Jul12 177:42 [vpthread-1-2]
root    3829   3.0  0.0      0      0 ?        S    Jun27 730:04 [rp_thread 7:0]
root    3826   3.0  0.0      0      0 ?        S    Jun27 728:08 [rp_thread 6:0]
[root@r6181-d5-us01 ~]# ls -l /usr/lpp/mmfs/bin/mmfdsd
-r-x----- 1 root root 20667161 Jun  3  2015 /usr/lpp/mmfs/bin/mmfdsd
[root@r6181-d5-us01 ~]#

```

GUI:

- User-friendly **desktop** metaphor interface
 - Usually mouse, keyboard, and monitor
 - **Icons** represent files, programs, actions, etc
 - Various mouse buttons over objects in the interface cause various actions (provide information, options, execute function, open directory (known as a **folder**))
- Many systems now include both CLI and GUI interfaces
 - Microsoft Windows is GUI with CLI “command” shell
 - Apple Mac OS X is “Aqua” GUI interface with UNIX kernel underneath and shells available
 - Unix and Linux have CLI with optional GUI interfaces (CDE, KDE, GNOME)

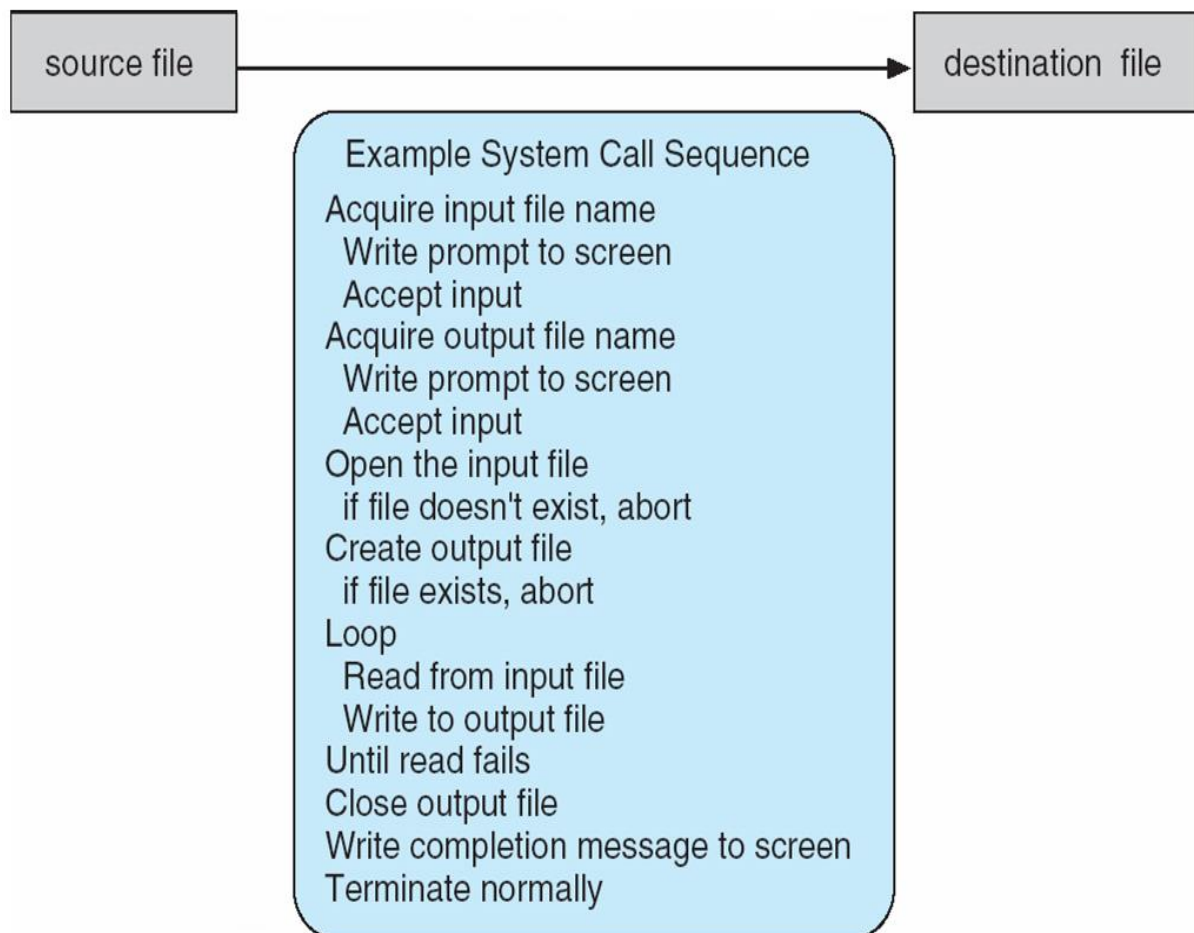
Touchscreen devices:

Touchscreen devices require new interfaces

- Mouse not possible or not desired
- Actions and selection based on gestures
- Virtual keyboard for text entry
- Voice commands

3.System Calls:

- System calls provide an interface to the services made available by an operating system.
- Typically these calls written in a high-level language (C or C++)
- Mostly accessed by programs via a high-level **Application Programming Interface (API)** rather than direct system call use.
- Three most common APIs are Win32 API for Windows, POSIX API for POSIX-based systems (including virtually all versions of UNIX, Linux, and Mac OS X), and Java API for the Java virtual machine (JVM).
- A programmer accesses an API via a library of code provided by the operating system.
- writing a simple program to read data from one file and copy them to another file.
- One approach is to pass the names of the two files as part of the command—for example, the UNIX cp command:\$cp in.txt out.txt.
- A second approach is for the program to ask the user for the names.
- System call sequence to copy the contents of one file to another file.



4.Types of System Calls:

Process control

- create process, terminate process
- end, abort
- load, execute
- get process attributes, set process attributes
- wait for time
- wait event, signal event
- allocate and free memory
- Dump memory if error
- **Debugger** for determining **bugs, single step** execution
- **Locks** for managing access to shared data between processes

File management:

- create file, delete file
- open, close file
- read, write, reposition
- get and set file attributes

Device management

- request device, release device
- read, write, reposition
- get device attributes, set device attributes
- logically attach or detach devices

Information maintenance

- get time or date, set time or date
- get system data, set system data
- get and set process, file, or device attributes

Communications

- create, delete communication connection
- send, receive messages if **message passing model** to **host name** or **process name**
 - From **client** to **server**
- **Shared-memory model** create and gain access to memory regions
- transfer status information
- attach and detach remote devices

Protection

- Control access to resources
- Get and set permissions
- Allow and deny user access

Examples of Windows and Unix System Calls

EXAMPLES OF WINDOWS AND UNIX SYSTEM CALLS

The following illustrates various equivalent system calls for Windows and UNIX operating systems.

	Windows	Unix
Process control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File management	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device management	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communications	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shm_open() mmap()
Protection	SetFileSecurity() InitializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()

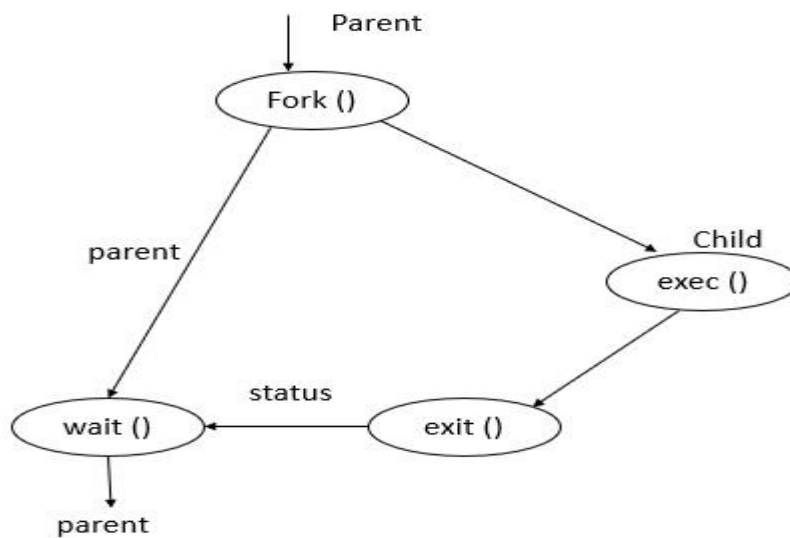
Process management system calls in Linux.

fork() – A parent process always uses a fork for creating a new child process. The child process is generally called a copy of the parent. After execution of fork, both parent and child execute the same program in separate processes.

exec() – This function is used to replace the program executed by a process. The child sometimes may use exec after a fork for replacing the process memory space with a new program executable making the child execute a different program than the parent.

exit() – This function is used to terminate the process.

wait() – The parent uses a wait function to suspend execution till a child terminates. Using wait the parent can obtain the exit status of a terminated child.



The systems calls for file management:

open – for opening the file for reading, writing.

close – to close the opened file.

read – for reading the data from the file into the buffer.

write – for writing the data from the buffer into file.

Now let us see about each system call in detail. We know a file is used to store a large volume of data. A file is treated as a single entity.

The main functions are as follows –

Create a file

Whenever you want to work on a file, the very first step is to create a file. A file is a space in memory where data is stored.

You can follow the steps to create a file –

Right click anywhere in the folder where you want to create a file.

Select new file.

Name the file and a file will be created.

Open a file

After creating a file, you can work on it whenever you want to and open it from the memory of the file system by accessing the path of the file.

For example – `cd ./user/file1` -----file1 being the name of the file that you created. Also you can double click on the file to open it wherever you have created it.

Close a file

You can close a file by clicking on the close button in the toolbar of a file.

Delete a file

Delete a file by following these steps –

Right click on the file.

Click on delete.

Click yes/confirm on the confirmation block.

5.System programs:

- System programs provide a convenient environment for program development and execution. They can be divided into:
 - File management
 - Status information
 - File Modifications
 - Programming language support
 - Program loading and execution
 - Communications
 - Background services
- Most users' view of the operating system is defined by system programs, not the actual system calls.

Some of them are simply user interfaces to system calls; others are considerably more complex.

- **File management** – These Programs Create, delete, copy, rename, print, dump, list, and generally manipulate files and directories

Status information

- Some ask the system for info - date, time, amount of available memory, disk space, number of users
- Others provide detailed performance, logging, and debugging information
- Typically, these programs format and print the output to the terminal or other output devices
- Some systems implement a **registry** - used to store and retrieve configuration information

File modification

- Several text editors may be available to create and modify the content of files stored on disk or other storage devices.
- There may also be special commands to search contents of files or perform transformations of the text.

Programming-language support - Compilers, assemblers, debuggers, and interpreters for common programming languages (such as C, C++, Java, and Python) are often provided with the operating system

Program loading and execution- Once a program is assembled or compiled, it must be loaded into memory to be executed. The system may provide Absolute loaders, relocatable loaders, linkage editors, and overlay-loaders, debugging systems for higher-level and machine language.

Communications - Provide the mechanism for creating virtual connections among processes, users, and computer systems

- Allow users to send messages to one another's screens, browse web pages, send electronic-mail messages, log in remotely, transfer files from one machine to another

Background Services

- Launch at boot time
 - ▶ Some for system startup, then terminate
 - ▶ Some from system boot to shutdown
- Provide facilities like disk checking, process scheduling, error logging, printing
- Run in user context not kernel context Known as **services**, **subsystems**, **daemons**

6.Operating System Design and Implementation:

- Design and Implementation of OS is not “solvable”, but some approaches have proven successful
- Internal structure of different Operating Systems can vary widely
- Start the design by defining goals and specifications
- Affected by choice of hardware, type of system
- **User goals and System goals**
 - User goals – operating system should be convenient to use, easy to learn, reliable, safe, and fast
 - System goals – operating system should be easy to design, implement, and maintain, as well as flexible, reliable, error-free, and efficient
- Specifying and designing an OS is highly creative task of **software engineering**

Policy: What needs to be done?

Example: Interrupt after every 100 seconds

Mechanism: How to do something?

Example: timer

Important principle: separate policy from mechanism

The separation of policy from mechanism is a very important principle, it allows maximum flexibility if policy decisions are to be changed later.

Example: change 100 to 200

Implementation

- Much variation
 - Early OSes in assembly language
 - Then system programming languages like Algol, PL/1
 - Now C, C++
- Actually usually a mix of languages
 - Lowest levels in assembly
 - Main body in C
 - Systems programs in C, C++, scripting languages like PERL, Python, shell scripts
- More high-level language easier to **port** to other hardware
 - But slower
- **Emulation** can allow an OS to run on non-native hardware

7.Operating System structure:

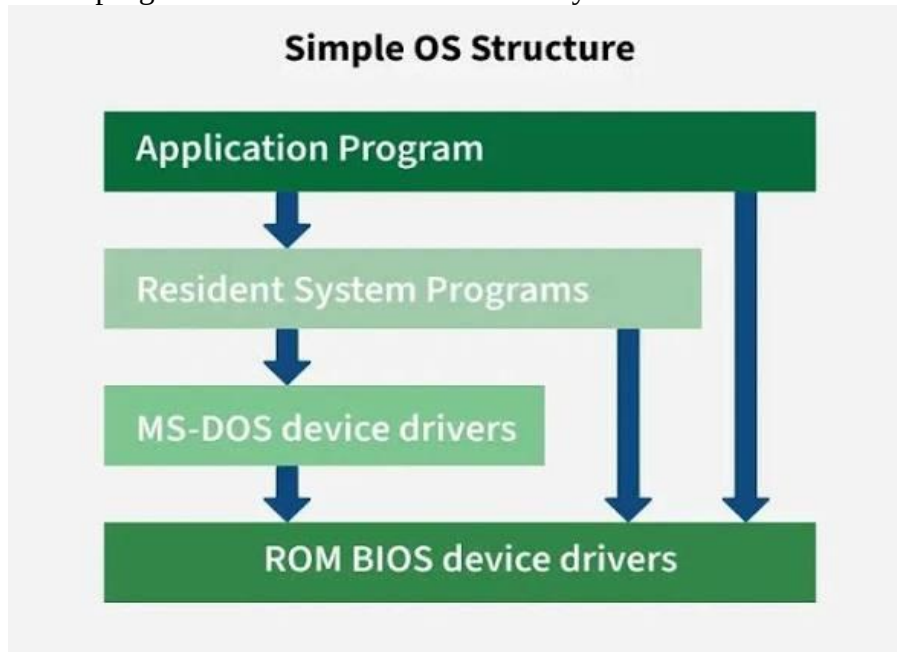
- General-purpose OS is very large program
- Various ways to structure ones
 - Simple Structure – MS-DOS
 - Monolithic Structure – UNIX
 - Layered – an abstraction

- Microkernel – Mach
- Module
- Hybrid

Simple structure – MS-DOS:

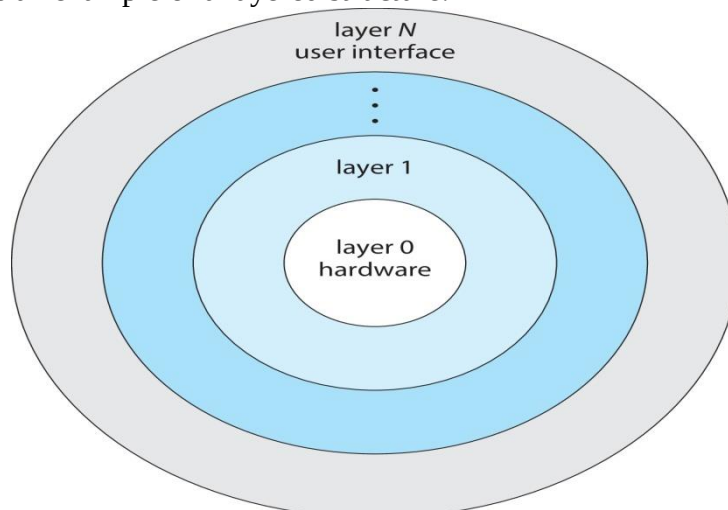
A basic, non-modular design where OS functions aren't well-separated, seen in early systems like MS-DOS, making it hard to maintain but simple for small systems.

- No well-defined structure; small and limited.
- Interfaces and functionality levels are not separated.
- MS-DOS is an example of this structure.
- Applications can directly access basic I/O routines.
- A user program failure can crash the entire system.



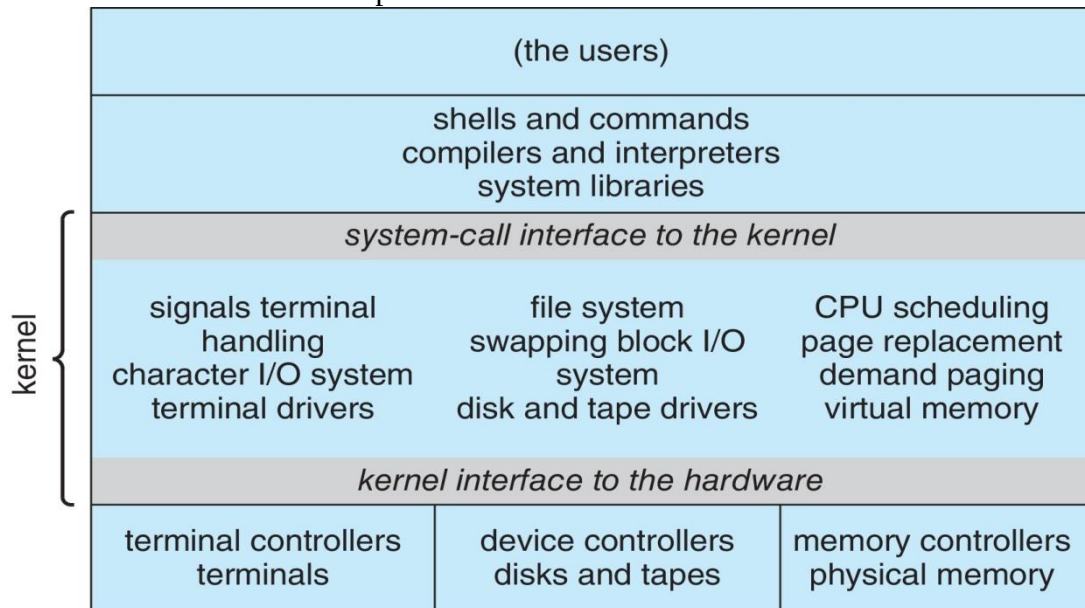
Layered Approach:

- OS is divided into multiple layers (hardware at bottom, UI at top).
- Each layer uses the functions of the layer below it.
- Easier debugging because errors are isolated to specific layers.
- Adds overhead due to data passing through many layers.
- Requires careful planning; layers can only use lower layers.
- UNIX is an example of a layered structure.



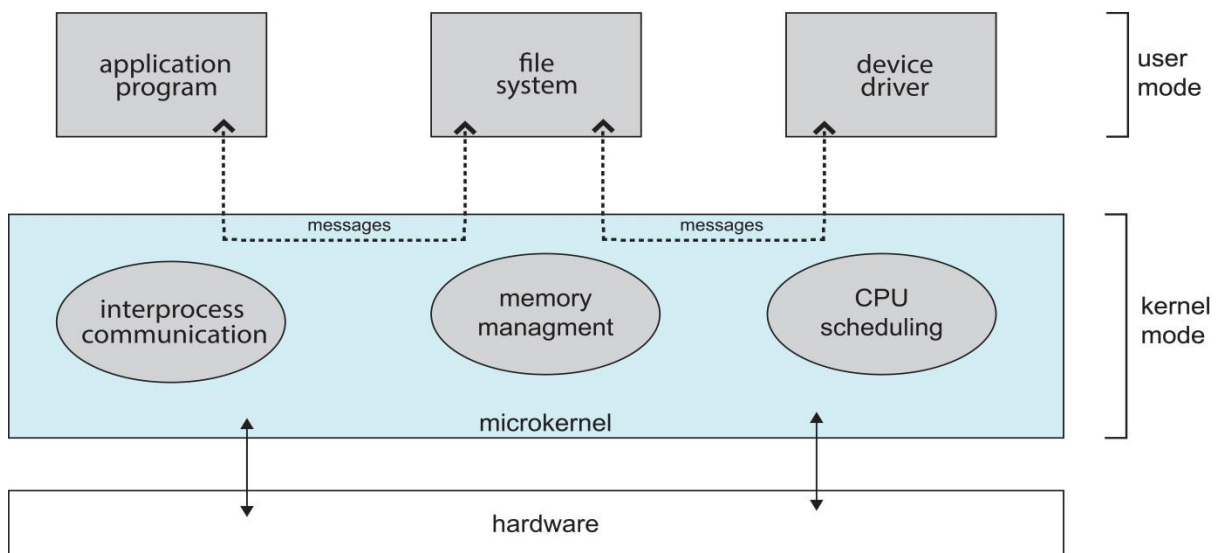
Monolithic Structure – Original UNIX

- Entire OS is implemented as one large kernel.
- Core services are combined into a single code base.
- Includes process, memory and file system management.
- Device drivers also operate inside the kernel.



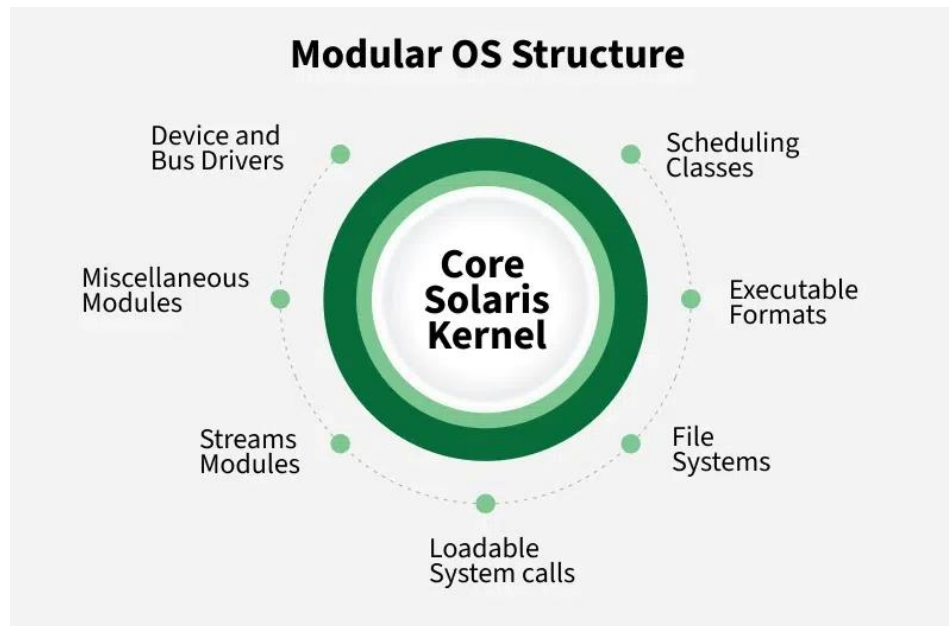
Microkernel System Structure

- Non-essential services run in user space, leaving a small core kernel.
- New services can be added without changing the kernel.
- More secure and reliable because failures don't crash the whole OS.
- macOS uses microkernel ideas within its hybrid XNU kernel.



Modules

- Kernel contains only core components.
- Extra services are added as dynamically loadable modules.
- More flexible than layered structure; modules can call any other module.
- Used in systems like Solaris.



Hybrid-Kernel Structure

- Combines monolithic and microkernel concepts.
- Provides monolithic-level speed and performance.
- Offers microkernel-style modularity and stability.
- Results in a more flexible and efficient OS design.

8. Building and Booting an Operating System:

- Operating systems generally designed to run on a class of systems with variety of peripherals
- Commonly, operating system already installed on purchased computer
 - But can build and install some other operating systems
 - If generating an operating system from scratch
 - ▶ Write the operating system source code
 - ▶ Configure the operating system for the system on which it will run
 - ▶ Compile the operating system
 - ▶ Install the operating system
 - ▶ Boot the computer and its new operating system
- Download Linux source code (<http://www.kernel.org>)
- Configure kernel via “make menuconfig”
- Compile the kernel using “make”
 - Produces vmlinuz, the kernel image
 - Compile kernel modules via “make modules”
 - Install kernel modules into vmlinuz via “make modules_install”
 - Install new kernel on the system via “make install”

System Boot

- When power initialized on system, execution starts at a fixed memory location
- Operating system must be made available to hardware so hardware can start it
 - Small piece of code – **bootstrap loader**, **BIOS**, stored in **ROM** or **EEPROM** locates the kernel, loads it into memory, and starts it
 - Sometimes two-step process where **boot block** at fixed location loaded by ROM code, which loads bootstrap loader from disk
 - Modern systems replace BIOS with **Unified Extensible Firmware Interface (UEFI)**

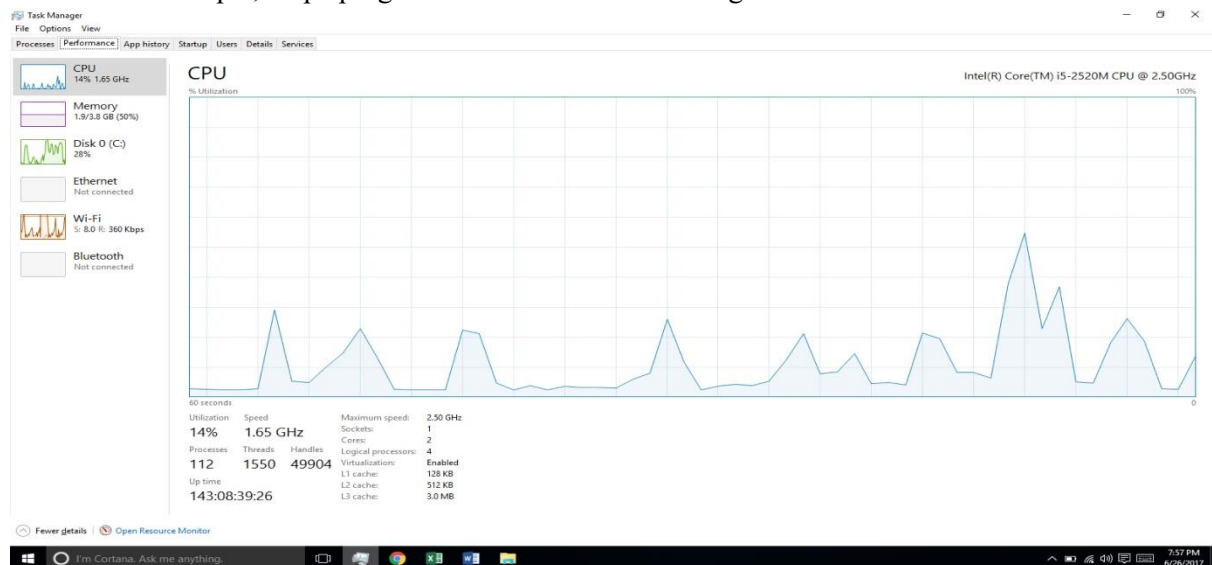
- Common bootstrap loader, **GRUB**, allows selection of kernel from multiple disks, versions, kernel options
- Kernel loads and system is then **running**
- Boot loaders frequently allow various boot states, such as single user mode

9. Operating System debugging:

- **Debugging** is finding and fixing errors, or **bugs**
- Also **performance tuning**
- OS generate **log files** containing error information
- Failure of an application can generate **core dump** file capturing memory of the process
- Operating system failure can generate **crash dump** file containing kernel memory
- Beyond crashes, performance tuning can optimize system performance
 - Sometimes using **trace listings** of activities, recorded for analysis
 - **Profiling** is periodic sampling of instruction pointer to look for statistical trends

Kernighan's Law: "Debugging is twice as hard as writing the code in the first place. Therefore, if you write the code as cleverly as possible, you are, by definition, not smart enough to debug it."

- Improve performance by removing bottlenecks
- OS must provide means of computing and displaying measures of system behavior
- For example, "top" program or Windows Task Manager



Tracing

Collects data for a specific event, such as steps involved in a system call invocation

Tools include

strace – trace system calls invoked by a process

gdb – source-level debugger

perf – collection of Linux performance tools

tcpdump – collects network packets

Debugging interactions between user-level and kernel code nearly impossible without toolset that understands both and can instrument their actions

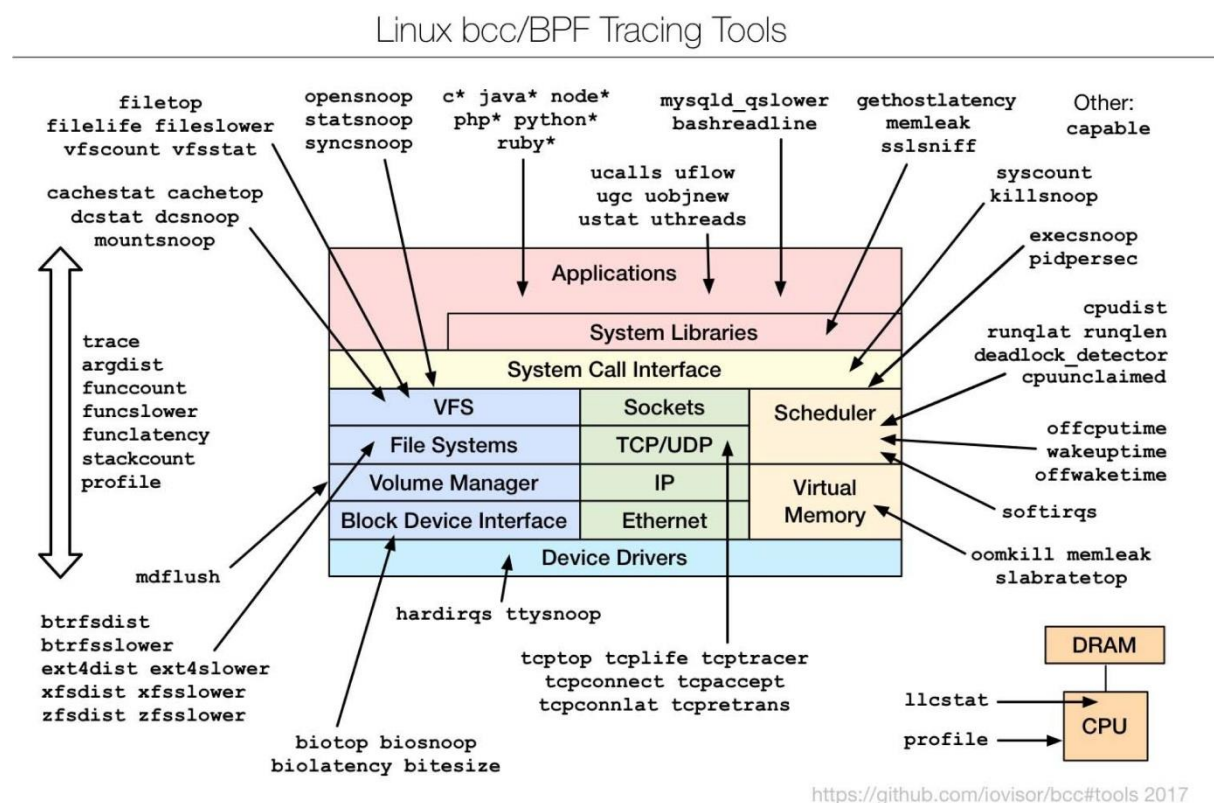
BCC:

BCC (BPF Compiler Collection) is a rich toolkit providing tracing features for Linux

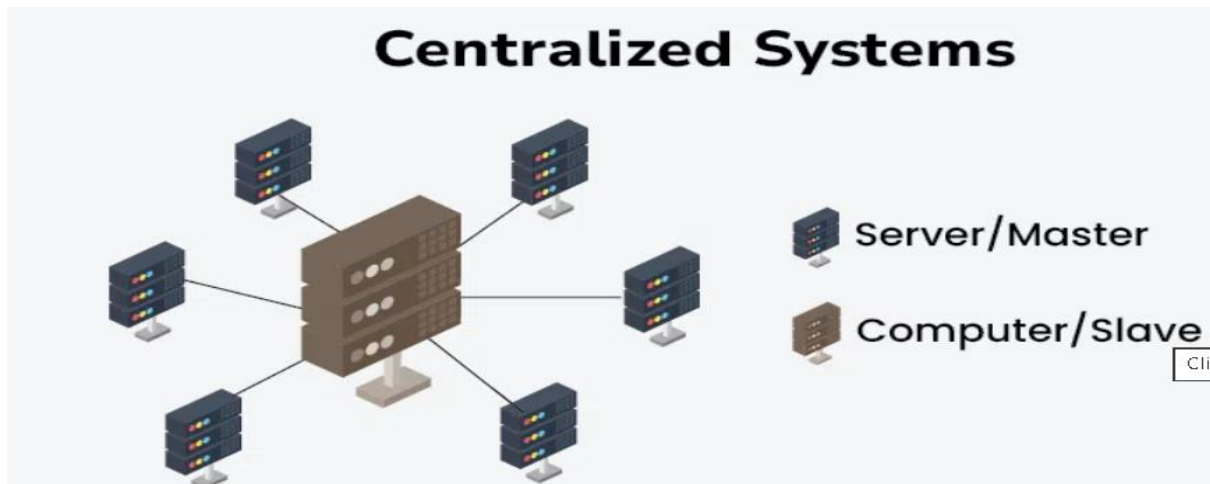
See also the original DTrace

For example, disksnoop.py traces disk I/O activity

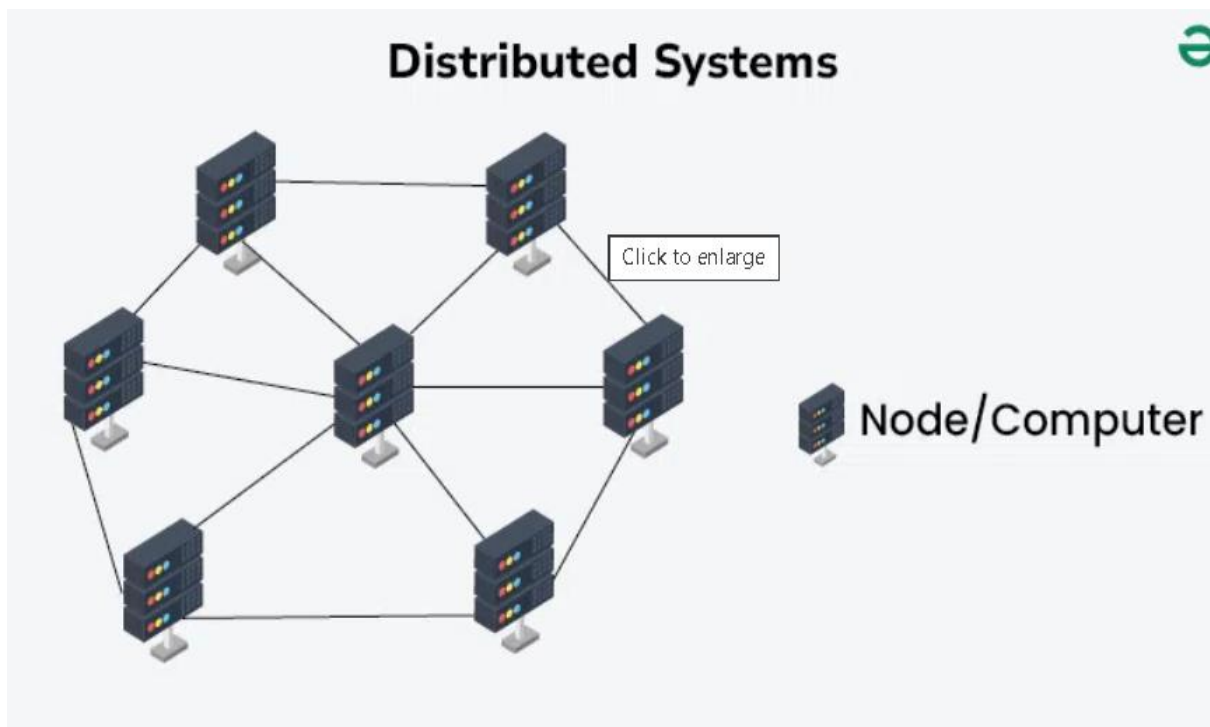
Linux bcc/BPF Tracing Tools



Centralized systems are a type of computing architecture where all or most of the processing and data storage is done on a single central server or a group of closely connected servers. This central server manages all operations, resources, and data, acting as the hub through which all client requests are processed. The clients, or nodes, connected to the central server typically have minimal processing power and rely on the server for most computational tasks.



Distributed systems are computing architectures where multiple independent nodes or computers work together to achieve a common goal. These nodes communicate and coordinate with each other over a network, appearing as a single coherent system to the end user. Distributed systems aim to improve performance, reliability, scalability, and resource sharing by leveraging the collective power of interconnected devices.



Differences between Centralized and Distributed Systems

Aspect	Centralized Systems	Distributed Systems
Definition	Single central server controls and manages all operations.	Multiple interconnected nodes working together as a single system.
Control	Centralized control with a single point of management.	Shared control, nodes collaborate to achieve common goals.
Single Point of Failure	High risk; if the central server fails, the whole system fails.	Reduced risk; designed for fault tolerance and redundancy.
Scalability	Limited scalability, can become a bottleneck.	Highly scalable, can add more nodes to distribute the load.
Resource Utilization	Central server resources are heavily utilized.	Efficient resource sharing across nodes.
Performance	Can be high initially but may degrade with increased load.	High performance due to parallel processing and resource sharing.
Management	Easier to manage centrally.	Complex, requires coordination and management of many nodes.
Latency	Lower latency, as operations are managed centrally.	Potentially higher latency due to network communication.